FIRST WORKING MODEL

People Counting with YOLO11 and ByteTrack

Interpretation: the four camera-separated sequences were used to compare detectors and choose the submitted model. The observed CA of 0.6548 is a post-selection result, not an unbiased final test estimate.

0.6548	1.7021	0.4113	0.4324
Observed mean CA	Curve MAE	MOTA	IDF1

Repository

https://github.com/NikolayTaran/cv_people_counting

Project objective

Count people entering and leaving a region under occlusion, crowding, and different camera views. The working system combines a YOLO11 person detector, custom ByteTrack tracker, and directional virtual-line counter.

Implemented architecture

Stage	Implementation	Output
Detection	YOLO11s, person class, confidence floor 0.05	Boxes and confidence
Tracking	Custom ByteTrack: Kalman + two-stage Hungarian IoU matching	Persistent track IDs
Counting	Foot-point crossing, hysteresis, cooldown, gap handling	Directional IN / OUT events
Evaluation	Predicted and GT trajectories use the same geometry	CA, curve MAE, MOTA, IDF1

1. Data loading and preprocessing

MOTChallenge images, seqinfo metadata, and gt/gt.txt annotations are loaded in frame order. Only valid class-1 pedestrian annotations are retained. Boxes are clipped to the image; boxes under 2 px and training annotations with visibility below 0.10 are removed. Remaining boxes become normalized YOLO labels.

Purpose	Sequences	Frames / images	Boxes
Train	First 80% of MOT17-04 and MOT17-05	1,510	39,144
Validation	Final 20% of MOT17-04 and MOT17-05	377	11,397
Model comparison	MOT17-02, MOT20-01, MOT17-09, MOT17-11	2,454	Not trained on
Untouched final test	None in this submission	-	-

The development split is chronological to reduce mixing of near-duplicate adjacent frames. The comparison cameras never enter the training exporter. The code calls this split 'test', but its results informed model choice; it is not an independent final test. VisDrone was unavailable and has no imputed result.

Preprocessing and augmentation

- Training: 640 px letterbox. Inference: 1280 px for small/dense scenes and 640 px for the others.
- Horizontal flip, translation, scale jitter, light mixup, and mosaic augmentation.

2. Model and training setup

YOLO11s starts from COCO weights and is fine-tuned with AdamW: lr0=0.001, cosine decay, weight decay 0.0005, batch 8, AMP, and seed 42. Training used an NVIDIA GeForce RTX 4060 Laptop GPU, PyTorch 2.11.0+cu128, and Ultralytics 8.4.175. Ultralytics chose the best checkpoint within each run on development validation fitness; the four comparison sequences were used to choose between detector configurations.

Checkpoint	Precision	Recall	mAP50	mAP50-95
3 epochs (submitted)	0.9712	0.8911	0.9636	0.6781
20 epochs	0.9644	0.9147	0.9643	0.7246

3. Observed comparison results

The same four sequences were used to compare four detectors and select the 3-epoch checkpoint. These figures describe that comparison; no untouched final test has yet been evaluated.

The primary comparison metric is mean counting accuracy (CA), averaged equally over the four sequences. Higher CA, MOTA, and IDF1 are better; lower curve MAE is better.

Detector	Mean CA	Curve MAE	MOTA	IDF1
YOLO11n, COCO pretrained	0.5501	2.5913	0.4361	0.4446
YOLO11s, COCO pretrained	0.6222	1.9747	0.4352	0.4671
YOLO11s, MOT fine-tuned 3 epochs	0.6548	1.7021	0.4113	0.4324
YOLO11s, MOT fine-tuned 20 epochs	0.5794	2.1542	0.4080	0.4277

On this comparison set, the submitted checkpoint scores 0.1047 CA above YOLO11n and 0.0326 above COCO-pretrained YOLO11s. Those observed differences do not estimate expected gains on new cameras without selection bias.

Submitted model by comparison sequence

Sequence	Pred IN/OUT	GT IN/OUT	CA	Curve MAE	MOTA	IDF1
MOT17-02	10 / 1	16 / 5	0.5238	2.2500	0.1797	0.2997
MOT20-01	7 / 2	21 / 5	0.3462	2.8750	0.4700	0.4433
MOT17-09	9 / 3	10 / 3	0.9231	0.1000	0.4770	0.4535
MOT17-11	9 / 10	11 / 12	0.8261	1.5833	0.5185	0.5331

Interpretation

- The short fine-tune has the highest observed CA on the comparison sequences.
- The 20-epoch model raises dense MOT20-01 CA from 0.4231 to 0.6154 but scores lower overall.
- Detection mAP and end-to-end counting quality differ: identity continuity near the line matters.

4. Problems and observations

- An initial 20-epoch run failed inside OpenCV warpAffine with Windows data-loader workers. It resumed from last.pt with workers=0 and completed.
- Development validation frames share the two training camera views, so high validation mAP did not guarantee the highest counting score across cameras.
- The four camera-separated sequences were used for model choice. Their 0.6548 CA must be read as a post-selection comparison result. For an independent final estimate, freeze the pipeline now and evaluate it once on new untouched cameras.
- The tracker lacks an appearance/ReID branch. Long occlusions can fragment or switch identities even with good detector recall.
- Counting geometry is configured per camera. Arbitrary new video would require calibration or automatic entrance-region estimation.

5. Reproduction

The repository includes the exporter, training script, submitted checkpoint, split-aware inference and evaluation, machine-readable results, and 39 passing unit tests. In the current CLI, '--split test' selects the four model-comparison sequences; the name does not make their result an untouched final test.

```
python scripts/prepare_training_data.py
```

```
python scripts/train_detector.py --epochs 3 --imgsz 640 --batch 8 --workers 0 --device 0
```

```
python scripts/run_pipeline.py --split test --weights models/people_yolo11s_mot.pt --device 0 --no-render --out-dir outputs/submission/runs
```

```
python scripts/evaluate.py --split test --runs-dir outputs/submission/runs --results outputs/submission/test_results.json
```

```
python -m pytest tests -q
```

Submission artifacts

- Submitted model: models/people_yolo11s_mot.pt
- Full methodology: docs/FIRST_WORKING_MODEL.md
- Machine-readable comparisons and training histories: reports/
- Repository: https://github.com/NikolayTaran/cv_people_counting